

Typeclasses from the viewpoint of type theory and category theory

For functional programmers, with Haskell examples

Sergei Winitzki

Academy by the Bay

2026-07-26

Typeclasses: an algebraic view

This material will appear in Chapter 13 of *The Science of Functional Programming*: <https://github.com/winitzki/sofp>

Plan:

- ❶ Examine the typeclass type signatures:
 - ▶ **algebraic** typeclasses ($P\ t \rightarrow t$)
 - ★ examples: Monoid, Functor, Monad, Applicative
 - ▶ **co-algebraic** typeclasses ($t \rightarrow P\ t$)
 - ★ examples: Show, Traversable
 - ▶ the Eq typeclass is neither algebraic nor co-algebraic
- ❷ Discover the universal patterns of typeclass derivation
- ❸ Define typeclasses as categories (with **typeclass-preserving** morphisms)
- ❹ Express typeclass laws via (co)monad algebra laws
- ❺ Prove the validity of the universal derivation patterns

Summary of typeclass derivation patterns

	Algebraic	Co-algebraic
class C t where	$m :: P\ t \rightarrow t$	$m :: t \rightarrow P\ t$
	instance C ()	instance C Void
instance (C a, C b) $\Rightarrow$	C (a, b)	C (Either a b)
instance (C a) $\Rightarrow$	C (r $\rightarrow$ a)	C (r, a)
instance (C x) $\Rightarrow$ C (F x)	C (Fix F)	C (Fix F)
(for any t)	instance (Free t)	instance (Cofree t)

- The new instances can be found for all typeclasses C
- The laws hold automatically for all new instances!
- Remarks:
 - ▶ P and F must be covariant functors (have lawful “fmap” methods)
 - ▶ Free and Cofree must be defined separately for each typeclass C
 - ▶ Certain typeclasses C support other, specific derivation patterns

Typeclass method signatures: examples I

- Gather all typeclass methods into one type signature
- See if that signature is algebraic or co-algebraic

- Example: the Monoid typeclass

```
class Monoid t where
  mappend :: t → t → t
  mempty  :: t
```

- ▶ uncurry the type of mappend as $(t, t) \rightarrow t$
- ▶ replace the type of mempty by $() \rightarrow t$
- ▶ combine into one function: $\text{Maybe } (t, t) \rightarrow t$
- ▶ this has the form $P\ t \rightarrow t$, where $P\ x = \text{Maybe } (x, x)$
- ▶ so, the Monoid typeclass is **algebraic**
- Terminology borrowed from category theory:
 - ▶ a “ P -functor algebra” is a type t with a function of type $P\ t \rightarrow t$
 - ▶ a “ P -functor coalgebra” is a type t with a function of type $t \rightarrow P\ t$
 - ▶ the typeclass methods are gathered into a single **structure map**

Typeclass method signatures: examples II

The Functor typeclass

- The typeclass applies to type *constructors*:

class Functor f where

fmap :: (a → b) → f a → f b

- ▶ uncurry the type: $(a \rightarrow b, f\ a) \rightarrow f\ b$
- ▶ this has the form $P\ f\ b \rightarrow f\ b$, where we define P by:
 $P\ f\ x = (a \rightarrow x, f\ a)$

★ note that the type parameter “a” is used existentially

- The Functor typeclass is **algebraic**

- The Functor typeclass is also **co-algebraic**:

- ▶ flip the curried arguments: $f\ a \rightarrow (a \rightarrow b) \rightarrow f\ b$
- ▶ this is of the form $f\ a \rightarrow P\ f\ a$, where we define P by:
 $P\ f\ x = (x \rightarrow b) \rightarrow f\ b$

★ the type parameter “b” is used existentially

★ the type parameter “x” is in a *covariant* position at right

- The (co)algebras are at the level of *type constructors*

- ▶ Instead of $P\ t \rightarrow t$ we have $\forall x. P\ f\ x \rightarrow f\ x$

Typeclass method signatures: examples III

- The Show typeclass:

```
class Show t where  
  show :: t → String
```

- ▶ The type signature has the form $t \rightarrow P\ t$, where P is defined by:
P x = String -- a constant functor
- ▶ The Show typeclass is **co-algebraic**

- The Traversable typeclass:

```
class (Applicative g) ⇒ Traversable f where  
  traverse :: (a → g b) → f a → g (f b)
```

- ▶ flip the curried arguments: $f\ a \rightarrow (a \rightarrow g\ b) \rightarrow g\ (f\ b)$
- ▶ this has the form $f\ a \rightarrow P\ f\ a$, where we define P by:
P f x = (x → g b) → g (f b)
 - ★ the type parameter “b” is used existentially
 - ★ the type parameter “x” is in a *covariant* position at right
- ▶ The Traversable typeclass is **co-algebraic**

Typeclass method signatures: examples IV

- The Pointed typeclass:

```
class Pointed f where  
  pure :: t → f t
```

- ▶ The type signature has the form $P f x \rightarrow f x$, where P is defined by:
 $P f x = x$ -- the identity functor
- ▶ The Pointed typeclass is **algebraic**

- The Eq typeclass:

```
class Eq t where  
  equal :: t → t → Bool
```

- ▶ The type signature is *not* of the form $P t \rightarrow t$ or $t \rightarrow P t$ with a covariant functor P
- ▶ The Eq typeclass is neither algebraic nor co-algebraic
- ▶ The theory developed here will mostly not apply to Eq
 - ★ Typeclass derivation for Eq will be *ad hoc*

Typeclasses, ADTs, and OO interfaces

ADT (algebraic data type)

```
data G where
```

```
  c1 :: Int → G
```

```
  c2 :: Bool → G → G
```

- Gather all constructors into a single type signature $P \ G \rightarrow G$, where P is defined by $P \ x = \text{Either } \text{Int} \ (\text{Bool}, x)$
- G is a type that supports a value $m :: P \ G \rightarrow G$

OO (object-oriented) interface

```
trait G:    // Scala 3
```

```
  def f1 : Boolean
```

```
  def f2(other: Int) : G
```

- Gather all methods into a single type signature $G \rightarrow P \ G$, where P is defined by $P \ x = (\text{Bool}, \text{Int} \rightarrow x)$
- G is a type that supports a value $m :: G \rightarrow P \ G$

Typeclass

- Type t belongs to the typeclass if there is a value $m :: Q \ t$

Derivation patterns for algebraic typeclasses I

- Given $p :: P\ a \rightarrow a$ and $q :: P\ b \rightarrow b$, what other types c can we construct such that we can derive $r :: P\ c \rightarrow c$?
 - ▶ Here P is viewed as an arbitrary fixed functor
 - ▶ We ignore the typeclass laws for now
- For $c = ()$, set r to the constant function of type $P\ () \rightarrow ()$
- For $c = d \rightarrow a$ (where d is any fixed type), write:
$$r :: P\ (d \rightarrow a) \rightarrow d \rightarrow a$$
$$r\ k\ x = p\ \$\ fmap\ (\backslash y \rightarrow y\ x)\ k$$
- For $c = (a, b)$, write:
$$r :: P\ (a, b) \rightarrow (a, b)$$
$$r\ k = (p\ x, q\ y)\ \text{where}$$
$$x = fmap\ fst\ k$$
$$y = fmap\ snd\ k$$
- But we cannot implement r for $c = \text{Either}\ a\ b$ in general
 - ▶ because we cannot map $P\ (\text{Either}\ a\ b) \rightarrow \text{Either}\ (P\ a)\ (P\ b)$
 - ★ although this is certainly possible for some P

Derivation patterns for algebraic typeclasses II

Recursive types (described via pattern functors)

- For a covariant functor F , the type c is a **fixpoint** of F if:

`fix :: F c → c`

`unfix :: c → F c`

`fix . unfix = id , unfix . fix = id`

- Assume that F “preserves the typeclass membership”: whenever a type t belongs to the typeclass then so does $F t$. This is expressed via the function:

`h :: (P t → t) → P (F t) → F t`

`h = ...` `-- assume that this function is known`

- Then c belongs to the typeclass:

`r :: P c → c`

`r = fix . h r . fmap unfix`

Derivation patterns for algebraic typeclasses III

Free typeclass instances

- A **free instance constructor** for an algebraic typeclass is a functor M such that:
 - ▶ For any type t , the type $M\ t$ belongs to the typeclass
 - ★ The structure map is $m :: P\ (M\ t) \rightarrow M\ t$
 - ▶ For any type c that belongs to the typeclass there is a unique typeclass-preserving function of type $M\ c \rightarrow c$
 - ★ The “typeclass-preserving” property will be explained shortly
 - ▶ There are some other technical conditions, omitted for brevity
- Examples:
 - ▶ Free Monoid is the `List` type constructor
 - ▶ Free Monad is the (higher-kinded) type constructor defined by:

```
data FreeMonad f a = Pure a | Join f (FreeMonad f a)
```
- When a typeclass has no laws, the free instance is `FreeMonad P`
- Otherwise, free instance constructors must be defined *ad hoc*

Derivation patterns for co-algebraic typeclasses I

- Given $p :: a \rightarrow P\ a$ and $q :: b \rightarrow P\ b$, what other types c can we construct such that we can derive $r :: c \rightarrow P\ c$?
 - ▶ Here P is an arbitrary fixed functor
 - ▶ We ignore the typeclass laws for now
- For $c = \text{Void}$, set r to the function `absurd` of type $\text{Void} \rightarrow P\ \text{Void}$
- For $c = (d, a)$ (where d is any fixed type), write:
 $r :: (d, a) \rightarrow P\ (d, a)$
 $r\ (k, x) = \text{fmap}\ (\backslash y \rightarrow (k, y))\ (p\ x)$
- For $c = \text{Either}\ a\ b$, write:
 $r :: \text{Either}\ a\ b \rightarrow P\ (\text{Either}\ a\ b)$
 $r = \backslash \text{case}$
 $\text{Left}\ x \rightarrow p\ x$
 $\text{Right}\ y \rightarrow q\ y$
- But we cannot implement r for $c = (a, b)$ in general
 - ▶ because we cannot always map $(P\ a, P\ b) \rightarrow P\ (a, b)$
 - ★ although this is certainly possible for some P

Derivation patterns for co-algebraic typeclasses II

Recursive types (described via pattern functors)

- For a covariant functor F , the type c is a **fixpoint** of F if:

`fix :: F c → c`

`unfix :: c → F c`

`fix . unfix = id , unfix . fix = id`

- Assume that F “preserves the typeclass membership”: whenever a type t belongs to the typeclass then so does $F t$. This is expressed via the function:

`h :: (t → P t) → F t → P (F t)`

`h = ...` `-- assume that this function is known`

- Then c belongs to the typeclass:

`r :: c → P c`

`r = fmap fix . h r . unfix`

Derivation patterns for co-algebraic typeclasses III

Cofree typeclass instances

- A **cofree instance constructor** for a co-algebraic typeclass is a functor N such that:
 - ▶ For any type t , the type $N\ t$ belongs to the typeclass
 - ★ The structure map is $m :: N\ t \rightarrow P\ (N\ t)$
 - ▶ For any type c that belongs to the typeclass there is a unique typeclass-preserving function of type $c \rightarrow N\ c$
 - ▶ There are some other technical conditions, omitted for brevity
- Examples:
 - ▶ Cofree Show is:
`type CofreeShow t = (t, String)`
 - ▶ Cofree Comonad is defined by (must use lazy pair!):
`data CofreeComonad f x = (x, f (CofreeComonad f x))`
- When a typeclass has no laws, CofreeComonad P works
- Otherwise, cofree instance constructors must be defined *ad hoc*

Typeclass-preserving functions: Algebraic case. I

- Take an algebraic typeclass whose structure map has type $P\ t \rightarrow t$. (Here P is a covariant functor.)

```
class C t where  
  m :: P t → t
```

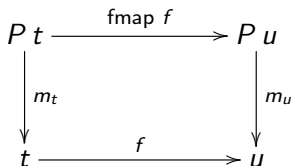
- Consider two types t and u that belong to the typeclass
- We say that a function $f :: t \rightarrow u$ is **typeclass-preserving** if f always maps the operations on t into the corresponding operations on u
- Example: the Monoid typeclass. A function f is “monoid-preserving” if it maps both operations from the monoid t to the monoid u
- The operation mappend:
 - ▶ The expression `mappend t1 t2` produces a new value of type t
 - ▶ Apply f to that value, obtain a value of type u
 - ▶ That value should be always equal to `mappend (f t1) (f t2)`
- The operation mempty:
 - ▶ The expression “`mempty`” is a value of type t
 - ▶ Apply f to that value, obtain a value of type u
 - ▶ That value should be equal to `mempty` for the type u

Typeclass-preserving functions: Algebraic case. II

Formulate the “typeclass-preserving” property in terms of P alone

- Example: the `Monoid` typeclass with $P\ t = \text{Maybe}\ (t, t)$ and the structure map $m_t :: P\ t \rightarrow t$.
 - ▶ The function “`mappend`” corresponds to the structure map m_t applied to a value of the form `Just (t1, t2)`
 - ▶ The function “`mempty`” corresponds to m_t applied to `Nothing`
 - ▶ The substituted arguments in `Just (f t1, f t2)` correspond to applying `(fmap f)` to the value `Just (t1, t2)`
 - ▶ Applying `(fmap f)` to `Nothing` produces again `Nothing`
- The typeclass-preserving condition becomes: for any s of type $P\ t$:

$$m_u (\text{fmap } f\ s) = f\ (m_t\ s) \quad \text{or:} \quad f \circ m_t = m_u \circ \text{fmap } f$$

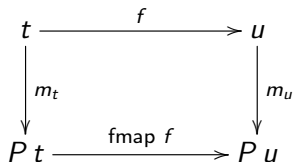


- ▶ In category theory, this is called the **P -algebra morphism law**

Typeclass-preserving functions: Co-algebraic case

- Take a co-algebraic typeclass with methods described as $t \rightarrow P\ t$
class C t where
 $m :: t \rightarrow P\ t$
- Let t, u be two types belonging to this typeclass, with structure maps m_t and m_u
- A function $f : t \rightarrow u$ is **typeclass-preserving** if:

$$\text{fmap } f \circ m_t = m_u \circ f$$



- Example: a Show-preserving function; $P\ t = \text{String}$

$$\text{show } (f\ x) = \text{show } x$$

- In other words, f preserves the result of applying `show`

Typeclass-preserving functions: General case I

- Gather all methods of a typeclass into a profunctor Q :
class C t where
 m :: Q t t
- Q is *contravariant* in the first type parameter and *covariant* in the second; has $\text{dimap} :: (a \rightarrow b) \rightarrow (c \rightarrow d) \rightarrow Q\ b\ c \rightarrow Q\ a\ d$
- Instead of a structure map, we have a typeclass instance value m
- If types t and u belong to this typeclass, a function $f :: t \rightarrow u$ is **typeclass-preserving** if the instance values m_t and m_u satisfy the “wedge condition”:

$$\begin{array}{ccc} Q\ t\ t & \xrightarrow{\text{dimap id } f} & Q\ t\ u \\ & & \uparrow \text{dimap } f\ \text{id} \\ & & Q\ u\ u \end{array}$$

$$\text{dimap id } f\ m_t = \text{dimap } f\ \text{id } m_u$$

Typeclass-preserving functions: General case II

- As an example, take the Eq typeclass
- Gather all methods of a typeclass into a **profunctor** Q :

```
type Q a b = (a, a) → Bool  
dimap :: (a → b) → (c → d) → Q b c → Q a d  
dimap f g q = \ (x, y) → q (f x, f y)
```
- The wedge condition for $f :: t \rightarrow u$ is: $m_u(f\ x, f\ y) = m_t(x, y)$
- In other words, f preserves the result of applying equal

Typeclass-preserving functions: Category laws

Typeclass-preserving functions are morphisms in a category

- Identity functions ($\text{id} :: t \rightarrow t$) are typeclass-preserving

$$\text{dimap id id} = \text{id}$$

- If $f :: t \rightarrow u$ and $g :: u \rightarrow v$ are typeclass-preserving then so is $g \circ f$

$$\begin{array}{ccccc} Q\ t\ t & \xrightarrow{\text{dimap id } f} & Q\ t\ u & \xrightarrow{\text{dimap id } g} & Q\ t\ v \\ & & \uparrow \text{dimap } f\ \text{id} & & \uparrow \text{dimap } f\ \text{id} \\ & & Q\ u\ u & \xrightarrow{\text{dimap id } g} & Q\ u\ v \\ & \searrow \text{dimap id } (g \circ f) & & \searrow \text{dimap } g\ \text{id} & \\ Q\ t\ v & \xleftarrow{\text{dimap } (g \circ f)\ \text{id}} & & & Q\ v\ v \end{array}$$

$$\text{dimap id } (g \circ f)\ m_t = \text{dimap } (g \circ f)\ \text{id}\ m_v$$

Typeclass-preserving functions: Proof of composition law

We have values m_t , m_u , m_v such that:

$$\text{dimap id } f \, m_t = \text{dimap } f \, \text{id } m_u \quad \text{and} \quad \text{dimap id } g \, m_u = \text{dimap } g \, \text{id } m_v$$

We need to show:

$$\text{dimap id } (g \circ f) \, m_t \stackrel{?}{=} \text{dimap } (g \circ f) \, \text{id } m_v$$

Left-hand side:

$$\begin{aligned} \text{dimap id } (g \circ f) \, m_t &= \text{dimap id } g \, (\text{dimap id } f \, m_t) \\ &= \text{dimap id } g \, (\text{dimap } f \, \text{id } m_u) = \text{dimap } f \, g \, m_u \end{aligned}$$

Right-hand side:

$$\begin{aligned} \text{dimap } (g \circ f) \, \text{id } m_v &= \text{dimap } f \, \text{id } (\text{dimap } g \, \text{id } m_v) \\ &= \text{dimap } f \, \text{id } (\text{dimap id } g \, m_u) = \text{dimap } f \, g \, m_u \end{aligned}$$

Typeclass laws I: Example

Typeclass laws are equations that the methods must satisfy

- Example: monoid laws

$$\begin{aligned} \text{mappend mempty } x &= x \quad , \quad \text{mappend } x \text{ mempty} = x \quad , \\ \text{mappend (mappend } x \ y) \ z &= \text{mappend } x \ (\text{mappend } y \ z) \end{aligned}$$

or in another notation:

$$\begin{aligned} e \oplus x &= x \quad , \quad x \oplus e = x \quad , \\ (x \oplus y) \oplus z &= x \oplus (y \oplus z) \end{aligned}$$

- Generally, typeclass laws are one or more equations for the structure map, using a number of arbitrary input values
- It turns out that these equations can be replaced by more general laws

Typeclass laws II: monad algebras

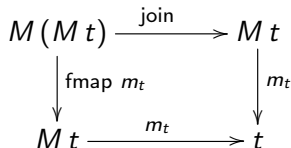
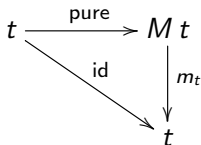
If M is a monad with methods:

$$\text{pure} :: a \rightarrow M a$$
$$\text{join} :: M (M a) \rightarrow M a$$

then an M -**monad algebra** is a type t and a structure map $m_t :: M t \rightarrow t$ satisfying the following identity and composition laws:

$$m_t \circ \text{pure} = \text{id}$$

$$m_t \circ \text{join} = m_t \circ (\text{fmap } m_t)$$



Typeclass laws III: deriving from monad algebras

Often, typeclass laws can be expressed via monad algebras

Example: how to derive Monoid's laws from List-monad algebra laws

- The List monad: $\text{pure } x = [x]$ and join is list concatenation
- Suppose t is a List-monad algebra, with $m :: \text{List } t \rightarrow t$
 - ▶ Denote $e = m []$ and $x \oplus y = m [x, y]$
 - ▶ The identity law says: $m [x] = x$
 - ▶ Use the composition law with argument $[x], []$
 - ★ $m (\text{join } [[x], []]) = m [x] = x$
 - ★ $m (\text{fmap } m [[x], []]) = m [x, e] = x \oplus e$
 - ▶ It follows that $x = x \oplus e$ and similarly $x = e \oplus x$
 - ▶ Use the composition law with arguments $[x, y], [z]$
 - ★ $m (\text{join } [[x, y], [z]]) = m [x, y, z]$
 - ★ $m (\text{fmap } m [[x, y], [z]]) = m [x \oplus y, z] = (x \oplus y) \oplus z$
 - ★ So, $(x \oplus y) \oplus z = m [x, y, z] = x \oplus (y \oplus z)$
 - ▶ All other instances of the composition law will now hold
- We have derived Monoid's laws
 - ▶ But we needed ingenuity to transform the laws into the familiar form

Typeclass laws IV: algebraic typeclasses

- If t is a `List`-monad algebra then t is a `Monoid`
- Generalize: for some monad M , if t is an M -monad algebra then t belongs to some *algebraic* typeclass (with methods $m :: P\ t \rightarrow t$)
 - ▶ To which typeclass? (What functor is P ?)
 - ▶ Given an algebraic typeclass, how to find a suitable monad M ?
- **Structure theorems** for algebraic typeclasses:
 - 1 If M is the *free instance constructor* of a given algebraic typeclass then M is a monad.
 - 2 With that M : If t is an M -monad algebra then t belongs to the typeclass.
 - 3 If an algebraic typeclass has no laws then $M = \text{FreeMonad } P$
- It is not obvious how to find M given an algebraic typeclass with a given structure functor P and a given set of laws
 - ▶ The free instance constructor has to be derived separately for every algebraic typeclass
 - ★ It is not clear that we will always find M within our type system
- It is also not obvious how to find P given M

Typeclass laws V: why M is always a monad

Consider an algebraic typeclass with structure functor P and the free instance constructor M

- For a typeclass member t we must have the “evaluation map”
 $M t \rightarrow t$
 - ▶ This is a unique typeclass-preserving function of this type
- Set $t = M u$ for any u , then we have a function $M (M u) \rightarrow M u$
 - ▶ This is M 's `join` method
- For any t we must have a function $t \rightarrow M t$
 - ▶ This is M 's `pure` method
- The monad laws of M follow from the properties of the evaluation map (with some technical assumptions omitted here)

Typeclass laws VI: laws and typeclass-preserving maps

Consider an algebraic typeclass with structure functor P and the free instance constructor M

- Typeclass-preserving maps are P -algebra morphisms
- All M -algebra morphisms are also P -algebra morphisms that obey all laws, and vice versa

An algebraic typeclass without laws = the category of P -functor algebras

An algebraic typeclass with laws = the category of M -monad algebras

- If $f :: t \rightarrow u$ is a surjective P -functor algebra morphism, and if a typeclass law holds in t , then that law also holds in u
- The evaluation map $M\ t \rightarrow t$ is a surjective typeclass-preserving map
 - ▶ It enforces the laws in t as long as they hold in $M\ t$
- If $f :: t \rightarrow u$ is a typeclass-preserving map and some law is violated in t but holds in u then the violations become invisible after applying f

Typeclass laws VII: co-algebraic typeclasses

- Co-algebraic typeclasses have methods $m :: t \rightarrow P\ t$
- The laws can be expressed via comonad coalgebras
- **Structure theorems** for co-algebraic typeclasses:
 - 1 If M is the *cofree instance constructor* of a given co-algebraic typeclass then M is a comonad.
 - 2 With that M : If t is an M -comonad coalgebra then t belongs to the typeclass.
 - 3 If a co-algebraic typeclass has no laws then $M = \text{CofreeComonad } P$
- It is not obvious how to find M given a co-algebraic typeclass with a given structure functor P and a given set of laws
 - ▶ The cofree instance constructor has to be derived separately for every co-algebraic typeclass
 - ▶ What is the cofree comonoid constructor? (I don't know.)
- It is also not obvious how to find P given M

Proving the algebraic typeclass derivation patterns I

The choice of the monad M encapsulates the full complexity of the laws
Assume that M is given, then it remains to prove that:

- 1 $t = ()$ is an M -monad algebra
- 2 If t is an M -monad algebra then so is $r \rightarrow t$ for any fixed type r
- 3 If a and b are M -monad algebras then so is (a, b)
- 4 If F is such that $F\ t$ is an M -monad algebra whenever t is, then any fixpoint of F is also an M -monad algebra

Proving the algebraic typeclass derivation patterns II

Example: how to prove this for $u = r \rightarrow t$

- Assume $m_t :: M\ t \rightarrow t$ is a given M -monad algebra
- We need to define $m_u :: M\ (r \rightarrow t) \rightarrow r \rightarrow t$

$m_u :: M\ (r \rightarrow t) \rightarrow r \rightarrow t$

$m_u\ mrt\ x = m_t\ \$\ fmap\ (\backslash y \rightarrow y\ x)\ mrt$

- Prove the M -monad algebra identity law for m_u , assuming that the law already holds for m_t :

$$\begin{aligned} (m_u\ .\ pure)\ f\ x &= m_u\ (pure\ f)\ x \\ &= m_t\ \$\ fmap\ (\backslash y \rightarrow y\ x)\ (pure\ f) \\ &= m_t\ \$\ pure\ (f\ x) \\ &= f\ x \end{aligned}$$

- Prove the M -monad algebra composition law for m_u :
 - ▶ (a longer derivation using naturality laws of M 's `join`, the functor composition laws, and the composition law of m_t)

Proving the co-algebraic typeclass derivation patterns

Assume that a comonad M is given, it remains to prove that:

- 1 $t = \text{Void}$ is an M -comonad coalgebra
- 2 If t is an M -comonad coalgebra then so is (r, t) for any fixed type r
- 3 If a and b are M -comonad coalgebras then so is $\text{Either } a \ b$
- 4 If F is such that $F \ t$ is an M -comonad coalgebra whenever t is, then any fixpoint of F is also an M -comonad coalgebra

- Most typeclasses are algebraic or co-algebraic
- Standard typeclass derivation combinators can be implemented as a library
 - ▶ The typeclass laws will hold automatically
- Free (co-free) typeclass instances need to be implemented separately
- Typeclass laws can be formulated via monad algebras or comonad coalgebras